CHAPTER 16: HUMAN REVIEW BY STAKES, NOT BY VOLUME

Summary: *Every organisation that adds human review to an automated system eventually discovers the same thing: reviewing by volume is unaffordable, and reviewing by convenience is unsafe. The axis that works is what actually goes wrong when a decision is wrong. This chapter covers how to route review by consequence rather than by size or timing, why the triggers have to be fixed before anyone sees a case, and the documented approval machinery that turns a policy into an actual gate.*

The Routing Axis Is Harm, Not Size

The question is which cases a person looks at, and there is exactly one defensible axis.

Route on what **actually goes wrong** when the **decision is wrong**. Three properties of that damage carry the choice:

Reversibility, meaning whether the wrong outcome can be undone at all, and what undoing it costs.

Repeatability, meaning whether one bad decision quietly recurs across every similar case until a human notices.

Who **bears** it, meaning whether the cost lands on the organisation that made the decision or on somebody who never agreed to any of it.

Now the two axes that feel right and are wrong.

Transaction size is not the axis. A large internal transfer that can be reversed in an afternoon carries less harm than a small message that reaches a customer and cannot be recalled. Routing by amount sends your reviewers confidently to the wrong queue, where they will find very little to catch.

The **savings the automation promises** is **not the axis** either. How much money the system saves has no relationship to what it costs when it is wrong, and using it as a routing input means the cases you most want reviewed are the ones you have most incentive to wave through.

Name the design: **review by stakes** sets each gate according to what a mistake there would cost, judged on whether the outcome can be undone and how far it spreads. What that rules out is worth stating explicitly, because both alternatives are common. A **single uniform posture applied to every action** is either too heavy to sustain or too light to protect. **After-the-fact reporting substituted for prevention** tells you what already happened to somebody, which is a different service entirely.

The affordability argument is the same argument. Pick the gate on whether the **outcome can be undone** and **how far it spreads**, instead of on **how much work** there is, **what time** it is, or **where the item sits** in a queue. That is the choice that makes **oversight worth having** and **possible to pay** for. Any scheme reviewing a fixed percentage is paying for coverage that it has no way of aiming at anything.

Triggers Fixed in Advance, Evaluated Against the Case

The second property of a working review design is about consistency, and it has a clean test.

Send the **same case through twice** and it has to **come out the same way**, whatever **hour** it lands and **whoever** happens to be working.

That takes conditions written down **before anybody sees a case**, and applied to what the **case actually contains**. Written down first, so nobody invents the rule while staring at the instance. Applied to the content, so routing follows properties of the work instead of the state of the queue.

Two schemes fail that test and both look reasonable on a slide. **Anything keyed to the clock**, reviewing the first hour of each day, or everything after a shift change, assumes the risky cases are spread evenly through time and they are not. **Anything keyed to queue position**, reviewing every tenth item, samples arrival order, which is unrelated to consequence.

The **named anti-pattern** is the **review queue as a sampling exercise**. It produces a percentage that sounds defensible in a meeting, alongside no relationship whatsoever between what gets reviewed and what carries consequence.

Two Design Decisions Nothing Else Can Supply

When a scenario asks what has to be settled before a pilot, two things are design decisions and the rest are their consequences.

Where the **oversight sits**, meaning which points in the flow have a gate at all.

What **governs it at each point**, meaning the rule that decides at each gate whether a case passes or stops.

Writing the process down and training the people who staff it are both real and both necessary, and both **follow from those two decisions**: you cannot draft a runbook or teach anybody until the gates exist and their checks are settled. And iterating on the design requires a pilot that has not yet run.

That is a general shape worth carrying past this chapter. A **design decision** is one that **determines what other work is possible**. An **operational readiness step** is work that **becomes possible** once the design exists. Scenarios in this area are built on the difference.

Permission Modes Are Stakes-Based Gating, Implemented

Claude Code's permission modes are a worked example of exactly this idea, and their details are examinable.[1]

Mode	What it runs without asking
Manual (config value default)	Reads only
acceptEdits	Reads, plus file edits and common filesystem commands such as mkdir, touch, mv, cp
auto	Everything, with background safety checks
dontAsk	Only pre-approved tools

Mode	What it runs without asking
bypassPermissions	Everything

Two of these repay attention. **dontAsk is not a permissive mode**, despite the name: it runs only what was pre-approved and simply declines to prompt about anything else. And** Manual carries the config value default**, which is the kind of naming mismatch a scenario can be built on.

Now the part that matters most, because it is the thing modes cannot override.

Some actions are **auto-approved by no mode** at all, including **bypassPermissions**.[1] Three categories:

- **Tools matched by an explicit ask rule**.
- **Connector tools an organisation has set to ask**.
- **Tools that require user interaction**, such as the built-in clarifying-question tool and MCP tools marked as requiring it.

An **organisational ask rule survives bypassPermissions**. That is the governance guarantee: the mode a developer chooses cannot dissolve a control the organisation set.

Plan Mode: Approval Before Anything Changes

Plan mode is review-by-stakes applied to the whole session rather than to individual actions. Claude researches and proposes changes without editing source, and **edits stay blocked until the plan is approved**.[1]

At the approval prompt there are three documented options: **approve and proceed in auto mode**, or auto-accept edits where auto mode is unavailable; **approve and manually approve each edit**; or **keep planning and say what to change**. And one detail worth knowing because it changes how the gate is used in practice: **Ctrl+G** opens the **proposed plan** in a **text editor** for **direct editing** before Claude proceeds.

That last option is the interesting one. The **gate is not only approve or reject**. A reviewer can rewrite the proposal and then approve their own version of it, which is a materially different control from a yes-or-no verdict on somebody else's plan.

The Approval Callback, and Waiting for a Human

Programmatic approval has documented mechanics that answer a family of scenarios about how a gate actually works at scale.[2]

A callback fires whenever Claude wants a tool that nothing earlier in the permission flow approved. It receives the tool name and its input, and **execution pauses until the callback returns**. The callback can **stay pending indefinitely**.

Then the fact that resolves the obvious objection: what if the human takes longer than the process can stay alive? A **deferred response** lets the **process exit** and **resume later** from the **persisted session**. So "the reviewer is on holiday" is not an architectural problem, it is a configuration one.

The available responses are richer than approve or deny, and knowing the list changes what designs are available:

Response	Effect
Approve as requested	The call runs as proposed
Approve with a modified input	The call runs with your changes. Claude **sees the result** but is **not told the input was changed**
Approve and suggest a permission rule	Matching calls skip the prompt next time; a suggestion carrying the local-settings destination is written to a settings file so future sessions skip it too
Reject with a message	Claude sees the message and may act on it

Response	Effect
Suggest an alternative	Redirects toward a different action
Redirect with new instructions	Replaces the intent entirely

The second row deserves care in a governance conversation. **Silently altering an input** is a **real capability** with a **real consequence**: the model observes an outcome that does not match what it asked for, and nothing tells it why. That is sometimes exactly the right thing to do, and it is always worth doing on purpose rather than by accident.

Two supporting details complete the picture. A **permission-request hook** can send **external notifications** to **Slack, email or push** when Claude is **waiting for approval**, which is what stops a pending gate from being an invisible stall. And the **clarifying-question tool** supports **1 to 4 questions** with **2 to 4 options** each, which bounds how much can be asked in one interruption.

What the Reviewer Needs to Decide Well

A gate is only as good as what arrives at it, and two documented capabilities are worth pairing here.

Citations return the exact passages supporting each claim, so a reviewer can verify an answer against its sources rather than forming an impression of it. One constraint matters: **citations** must be **enabled on all or none** of the documents in a request. And a useful economic note: the field carrying the **quoted passage** counts toward **neither output tokens nor input tokens**.[3] Attribution is not one of the token costs you are trading anything against.

Structured outputs constrain the **response to a schema** through **constrained decoding**, so downstream processing receives valid, parseable output rather than possibly malformed JSON.[4] For a review workflow that means the reviewer's queue can be built from fields rather than from parsed prose.

One more, from the response itself: the **tool_use stop reason** means Claude is **calling a tool** and **expects the caller to run** it.[5] That is the hook a gate hangs on, and reading stop reasons is how you know a decision point has arrived.

How Wrong Answers Are Built on Human Review Questions

- **Routing by size instead of by harm**. The test: does this scheme send the **irreversible small thing** to a reviewer, or only the **large reversible one**?
- **Routing by clock or queue position**. The test: would **two identical cases arriving at different times** get the same treatment?
- **Reporting offered as prevention**. The test: does this **stop the outcome**, or **describe it afterwards**?
- A **downstream product presented as a design decision**. The test: could this be **produced before the design exists**?
- A **uniform posture applied everywhere**. The test: is the gate the **same for a draft** and for an **irreversible external action**?

When the scenario says	The answer is usually	The trap is usually
Which cases should a human review	Route on whether it can be undone, whether it recurs, and who absorbs it	Transaction size, or the savings the automation promises
Review must be affordable and meaningful	Gate on whether it can be undone and how far it spreads	A fixed review percentage, which cannot aim
Consistency of review is questioned	Conditions written before any case arrives, applied to the case's content	Clock-based windows or every-Nth-item sampling
What must be decided before a pilot, two wanted	Where the gates sit, and what rule each one applies	Writing the process down, teaching the reviewers, or iterating on results nobody has

When the scenario says	The answer is usually	The trap is usually
An organisation must guarantee certain tools always prompt	An ask rule, which no mode overrides, including bypassPermissions	Relying on the session's permission mode
dontAsk is proposed as a permissive setting	It runs only pre-approved tools	Reading the name as "allow everything quietly"
Changes must be reviewed before any edit happens	Plan mode, with edits blocked until approval	Reviewing the diff after the edits are made
A reviewer disagrees with part of a proposed plan	Edit the plan directly, then approve	Treating approval as binary
A human may take hours to approve	A deferred response; the process exits and resumes from the persisted session	Assuming the process must stay alive
An approval is pending and nobody notices	A permission-request hook sending Slack, email or push	Waiting on somebody to check the terminal
A reviewer must verify an answer against sources	Citations, enabled on all documents in the request	Enabling them on some documents, which is not allowed
A review queue is built from model output	Structured outputs, so fields arrive parseable	Parsing prose

The habit worth carrying: for each gate, ask **what it costs to be wrong** on the **other side** of it. If the answer amounts to an apology, the gate has been placed somewhere it was never needed.

What to Remember

- Route human review on what a **wrong decision** does: whether **anyone can undo** it, whether it **recurs across similar cases**, and who **absorbs the cost**. **Transaction size is not the axis**, because a small irreversible external action outranks a large reversible internal one. The savings the automation promises is **not the axis either**.

- **Review by stakes** sets each gate according to what a **mistake there would cost**, judged on **whether the outcome can be undone** and how **far it spreads**. That is what makes oversight **meaningful and affordable at the same time**, and it rules out **one uniform posture everywhere** and **after-the-fact reporting standing in for prevention**.

- Conditions are **written down before anybody sees a case** and applied to what the **case contains**, so the **same case comes out the same way** whoever is working. **Clock-based windows** and **every-Nth-item sampling** both fail this, because neither is **related to consequence**.

- Two things are design decisions before a pilot: where the **gates sit**, and what **rule each one applies**. Writing it down and training the staff both **follow** from those, and iteration needs a **pilot nobody has run** yet.

- Permission modes: **Manual** (config value default) runs **reads only**; **acceptEdits** adds **file edits and common filesystem commands**; **auto** runs everything with **background safety checks**; **dontAsk** runs **only pre-approved tools**; **bypassPermissions** runs everything.

- Some actions are **auto-approved by no mode at all**, including bypassPermissions: tools matched by an **explicit ask rule**, connector tools an **organisation set to ask**, and tools **requiring user interaction**. An **organisational ask rule survives bypassPermissions**.

- In plan mode, **edits stay blocked until the plan is approved**. The approval options are **proceed in auto mode**, **approve each edit manually**, or **keep planning**, and **Ctrl+G opens the plan in an editor** so a reviewer can **rewrite it before approving**.

- The approval callback receives the **tool name and input**, **pauses execution until it returns**, and may **stay pending indefinitely**. A **deferred response** lets the process **exit and resume later** from the persisted session, so a **slow human is not an architectural problem**.

- Responses include **approving as requested**, **approving with a modified input**, **approving plus suggesting a permission rule**, **rejecting with a message Claude can act** on, **suggesting an alternative**, and **redirecting with new instructions**. When an input is modified, Claude **sees the result** but is **not told it changed**.

- A **permission-request hook** can notify **Slack, email or push** while Claude waits, and the clarifying-question tool allows **1 to 4 questions** with **2 to 4 options each**.

- Citations give the **exact supporting passages**, must be enabled on **all or none of the documents** in a request, and the quoted-passage field **counts toward neither input nor output tokens**.

- **Structured outputs constrain responses to a schema** by constrained decoding, so a review queue is built from **parseable fields instead of prose**. The **tool_use stop reason** means Claude **expects the caller to run a tool**, which is where a **gate attaches**.